SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO4: *Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:100 Duration : 1 Hour

Industry Problem

Code of Conduct:

I Shaik Mohammed Waseem Rayan (192373004) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Mohammed Waseem Rayan

1. Design and implement a C-based validation module that simulates a Deterministic Finite Automaton (DFA) to verify input strings in a real-time system. The module sh ould accept only those strings that begin with the character ‘a’ and end with the character ‘a’, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- **Process user input dynamically**
- **Validate strings based on DFA state transitions**
- **Output whether the given input is Accepted or Rejected**

DFA-Based String Validation in C

This program simulates a **Deterministic Finite Automaton (DFA)** that accepts only those strings that:

- Start with 'a'
- End with 'a'

The program processes user input dynamically, performs state transitions, and outputs **Accepted** or **Rejected**.

DFA Design

States:

- **q0** → Start state
- **q1** → String starts with a and current last character is a (**Accept State**)
- **q2** → String starts with a and current last character is not a
- **q3** → Dead/Reject state (string does not start with a)

Current State	Input = a	Input ≠ a
q0	q1	q3
q1	q1	q2
q2	q1	q2
q3	q3	q3

Transitions:

Accept State: **q1**

C Program

```
#include <stdio.h>
#include <string.h>

int main() { char
   str[100];
```

```
int state = 0;  // q0

printf("Enter a string: ");
scanf("%s", str);

for (int i = 0; i < strlen(str); i++) {
    char ch = str[i];

    switch (state) {
        case 0: // q0
            if (ch == 'a')
                state = 1; // q1
            else
                state = 3; // q3
            break;

        case 1: // q1
            if (ch == 'a')
                state =
                1;
            else
                state = 2; // q2
            break;

        case 2: // q2
            if (ch == 'a')
                state =
                1;
            else
                state = 2;
            break;

        case 3: // q3 (dead state)
            state = 3;
            break;
    }
}

if (state == 1)
    printf("Accepted\n");
else
    printf("Rejected\n");
```

```
    return 0;
}
```

Sample Execution
Input 1
Enter a string: aba
Output
Accepted

Input 2
Enter a string: abcda
Output
Accepted

Input 3
Enter a string: bacab
Output
Rejected

Time Complexity

- **O(n)**, where n is the length of the input string.
- The DFA scans each character exactly once.

This implementation efficiently validates whether a string **begins with 'a' and ends with 'a'** using DFA state transitions.

2. Design a Push Down Automata that accepts the language

$L = \{w \mid w \in (0+1)* \; and \; n_0(w) = n_1(w)\}$

$n_0(w)$ **is the number of 0's in w**

$n_1(w)$ **is the number of 1's in w**

Assuming the intended language is:

where:

- = number of 0s in string
- = number of 1s in string

The PDA accepts a string when the **number of 0s equals the number of 1s**, regardless of their order.

PDA Definition

The PDA is:

where:

- — set of states
- — input alphabet
- — stack alphabet
- — initial state
- — initial stack symbol
- — final state
- — set of final states

Working Principle

The PDA uses the stack to **cancel opposite symbols**:

- When input 0 is read:
    - If stack top is 1, pop 1.
    - Otherwise, push 0.
- When input 1 is read:
    - If stack top is 0, pop 0.
    - Otherwise, push 1.

Therefore, after processing the complete input:

- **Empty stack (apart from) → Accepted**
- **Remaining 0 or 1 symbols → Rejected**

Transition Table

Current State	Input	Stack Top	Stack Operation	Next State
q0	0	1	Pop 1	q0
q0	0	0	Push 0	q0
q0	0	Z_0	Push 0	q0
q0	1	0	Pop 0	q0
q0	1	1	Push 1	q0
q0	1	Z_0	Push 1	q0
q0	ε	Z_0	Move to final state	qf

Example 1: 0011
Number of 0s = 2
Number of 1s = 2
Stack operation:

Input: 0 → Push 0

Input: 0 → Push 0

Input: 1 → Pop 0

Input: 1 → Pop 0
Stack returns to .
Result: ACCEPTED
Example 2: 0101
Input: 0 → Push 0

Input: 1 → Pop 0

Input: 0 → Push 0

Input: 1 → Pop 0

Number of 0s = 2
Number of 1s = 2

Result: ACCEPTED

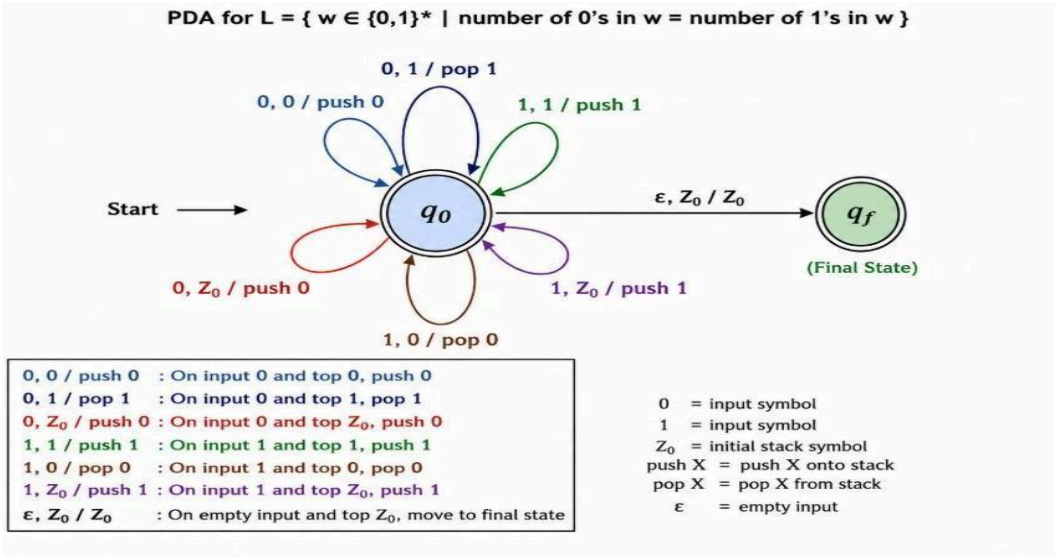
Example 3: 001

Input: 0 → Push 0

Input: 0 → Push 0

Input: 1 → Pop 0
One 0 remains in the stack.
Number of 0s = 2
Number of 1s = 1

Result: REJECTED



3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

L={a^n b^2n | n≥1}

The language contains **n as followed by exactly 2n bs**.

Examples

- abb → → **Accepted**
- aabbbb → → **Accepted**
- aaabbbbbb → → **Accepted**
- aabbb → → **Rejected**
- abab → **Rejected** because all as must come before bs.

PDA Idea

For every a read, the PDA pushes **two symbols X** onto the stack.

Then, for every b read, it pops **one X**.

Therefore:

and

The string is accepted when all input is consumed and the stack returns to the initial symbol .

PDA Components

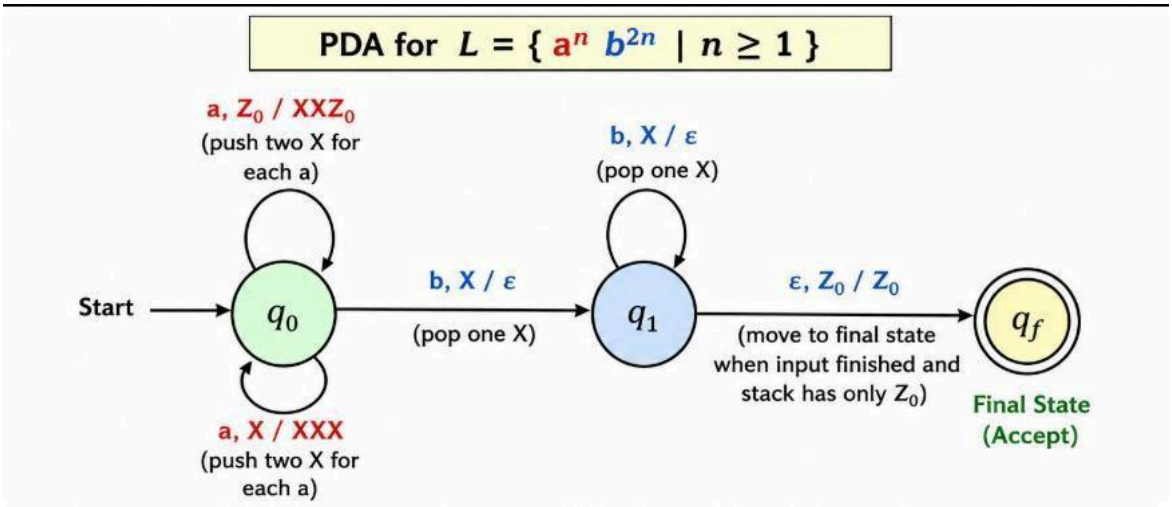
Where:

- = initial state
- = initial stack symbol
- = final state

Transition Table

Current State	Input	Stack Top	Stack Operation	Next State
	a		Push	
	a		Push	
	b		Pop	
	b		Pop	
			Keep	

State Diagram



Example: aabbbb

Initial stack: Read
first a: Read
second a: Now
read four bs:

$b \rightarrow XXXZ_0$

$b \rightarrow XXZ_0$

$b \rightarrow XZ_0$

$b \rightarrow Z_0$

Input is completely consumed and the stack contains only .

Therefore, aabbbb is ACCEPTED.

Conclusion

The PDA accepts strings of the form:

It ensures that **all as occur first** and that the number of bs is **exactly twice the number of as**.

4. Develop a C-based computational module to determine the ε-closure of all states in a Non Deterministic Finite Automaton (NFA) with ε-transitions, as part of an automata processing engine used in compiler construction and pattern-matching systems.

Aim

To develop a **C-based computational module** that calculates the **ε-closure of every state** in a Non-Deterministic Finite Automaton (NFA) containing ε-transitions.

The ε-closure of a state is the set of all states reachable from that state using **zero or more ε-transitions**.

Example

Consider the following ε-transitions:

q0 --ε--> q1

q1 --ε--> q2

q2 --ε--> q3

q2 --ε--> q4

Then:

ε-closure(q0) = {q0, q1, q2, q3, q4}

ε-closure(q1) = {q1, q2, q3, q4}

ε-closure(q2) = {q2, q3, q4}

ε-closure(q3) = {q3}

ε-closure(q4) = {q4}

C Program

```
#include <stdio.h>

#define MAX_STATES 20

int epsilon[MAX_STATES][MAX_STATES];

int visited[MAX_STATES];

int n;

/* Recursive function to find epsilon closure */

void epsilonClosure(int state)

{

    int i;


    visited[state] = 1;
```

```
    for (i = 0; i < n; i++)
    {
        if (epsilon[state][i] == 1 && visited[i] == 0)
        {
            epsilonClosure(i);
        }
    }
}
/* Display epsilon closure of a state */
void displayClosure(int state)
{
    int i;
  for (i = 0; i < n; i++)
    {
        visited[i] = 0;
    }
 epsilonClosure(state);
  printf("Epsilon-closure(q%d) = { ", state);
   for (i = 0; i < n; i++)
```

```
    {
        if (visited[i] == 1)
        {
            printf("q%d ", i);
        }
    }
    printf("}\n");
}
int main()
{
    int transitions;
    int from, to;
    int i;
  printf("Enter the number of states: ");
    scanf("%d", &n);
 /* Initialize transition matrix */
    for (i = 0; i < n; i++)
    {
        int j;
        for (j = 0; j < n; j++)
```

```
        {
            epsilon[i][j] = 0;
        }
    }
printf("Enter the number of epsilon transitions: ");
    scanf("%d", &transitions);

    printf("\nEnter epsilon transitions (from state to state):\n");

    for (i = 0; i < transitions; i++)
    {
        printf("Transition %d: ", i + 1);
        scanf("%d %d", &from, &to);

        epsilon[from][to] = 1;
    }
printf("\n--- Epsilon Closures ---\n"); /* Find epsilon closure for every state */
    for (i = 0; i < n; i++)
    {
        displayClosure(i);
```

```
    }

    return 0;

}
```

Sample Input

Enter the number of states: 5

Enter the number of epsilon transitions: 4

Enter epsilon transitions (from state to state):

Transition 1: 0 1

Transition 2: 1 2

Transition 3: 2 3

Transition 4: 2 4

Sample Output

--- Epsilon Closures ---

Epsilon-closure(q0) = { q0 q1 q2 q3 q4 }

Epsilon-closure(q1) = { q1 q2 q3 q4 }

Epsilon-closure(q2) = { q2 q3 q4 }

Epsilon-closure(q3) = { q3 }

Epsilon-closure(q4) = { q4 }

Algorithm

1. Read the number of states in the NFA.

2. Read the number of ε-transitions.
3. Store the ε-transitions in an adjacency matrix.
4. Select each state one by one.
5. Mark the selected state as visited.
6. Follow every available ε-transition recursively.
7. Mark each reachable state as visited.
8. Display all visited states as the ε-closure of the selected state.
9. Repeat the process for all states.

Result

The C-based module successfully computes the **ε-closure of all states** in an NFA. The recursive traversal ensures that all states reachable through one or more ε-transitions, along with the state itself, are included in the closure. This technique is useful in **NFA-to-DFA conversion, compiler construction, lexical analysis, and pattern-matching systems**.

5. Develop a C-based input validation module that determines whether a given binary string conforms to a specified Context-Free Grammar (CFG) used in structured data validation systems.

The grammar is defined as:

- **S → 0 A 1**
- **A → 0 A | 1 A | ε**

Aim

To develop a **C-based input validation module** that checks whether a given binary string conforms to the following Context-Free Grammar (CFG):
The grammar generates all binary strings that **start with 0 and end with 1**, with any combination of 0s and 1s in between.

Language Generated

Examples:
01 → Accepted

001 → Accepted
011 → Accepted
0101 → Accepted
001101 → Accepted
10 → Rejected
00 → Rejected
11 → Rejected

C Program

```
#include <stdio.h>
#include <string.h>

int validateCFG(char str[])
{
    int length = strlen(str);

    /* String must contain at least two characters */
    if (length < 2)
        return 0;

    /* S -> 0 A 1
       Therefore, first character must be 0
       and last character must be 1.
    */
    if (str[0] != '0' || str[length - 1] != '1')
        return 0;

    /* A -> 0A | 1A | epsilon
       Therefore, all characters in the
       middle must be either 0 or 1.
    */
    for (int i = 1; i < length - 1; i++)
    {
        if (str[i] != '0' && str[i] != '1')
            return 0;
    }

    return 1;
}

int main()
{
```

```
    char str[100];

    printf("Enter a binary string: ");
    scanf("%99s", str);

    if (validateCFG(str))
        printf("Accepted: String conforms to the CFG.\n");
    else
        printf("Rejected: String does not conform to the CFG.\n");

    return 0;
}
```

Sample Execution 1

Enter a binary string: 0101
Accepted: String conforms to the CFG.

Derivation

For 0101:
Therefore, **0101 is Accepted**.

Sample Execution 2

Enter a binary string: 001101
Accepted: String conforms to the CFG.
The string starts with 0 and ends with 1, and all intermediate characters are either 0 or 1.
Therefore, **Accepted**.

Sample Execution 3

Enter a binary string: 1001
Rejected: String does not conform to the CFG.
The string starts with 1, but the grammar requires:
Therefore, **Rejected**.

Algorithm

1. Read the binary string from the user.
2. Check whether the length is at least 2.
3. Check whether the first character is 0.
4. Check whether the last character is 1.
5. Verify that all characters between the first and last positions are either 0 or 1.
6. If all conditions are satisfied, display **Accepted**.
7. Otherwise, display **Rejected**.

Result

The C-based validation module successfully determines whether a given binary string belongs to

the language generated by the CFG:
Thus, the module accepts strings that **begin with 0, end with 1, and contain only binary symbols in between**.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand